

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_17

17. Chain of Responsibility Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Chain of Responsibility pattern.

GoF Definition

It avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. It chains the receiving objects and passes the request along the chain until an object handles it.

Concept

In this pattern, you form a chain of objects where you pass the responsibility of a task from one object to another and so forth until an object accepts the responsibility of completing the task. Each object in the chain can handle a particular kind of request. If an object cannot handle the request fully, it passes the request to the next object in the chain. This process may continue until the end of the chain. This kind of request-handling mechanism gives you the flexibility to add a new processing object (handler) or shuffle their order in the chain. Figure 17-1 shows such a chain with N number of handlers.

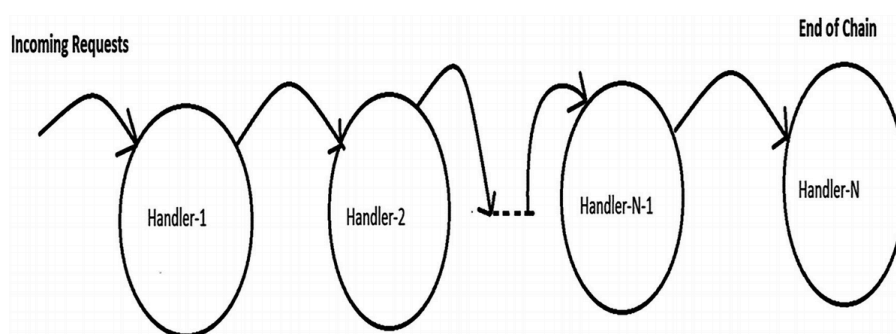


Figure 17-1 Chain of Responsibility pattern

Real-Life Example

Most software organizations have customer care representatives who take feedback from customers and forward any issues to the appropriate departments in the organization. However, not all of these departments will start fixing the issue simultaneously. The department that seems to be responsible will take a look at the issue first, and if those employees

believe that the issue should be forwarded to another department, they will forward it.

You may see a similar scenario when a patient visits a hospital. Doctors from one department can refer the patient to a different department for further diagnosis if they think it's needed.

You can consider a mobile company organization, too. For example, in India, the Vodafone mobile company runs a customer care department. If you have a complaint about the service, you first raise the issue to the customer care department. If they fail to solve your problem, you can escalate it to a nodal officer. If you are not satisfied with the solution given by the nodal officer, you can further escalate the issue to an appellate officer.

Computer World Example

Consider a software application (say, a printer) that can send e-mails and faxes. As a result, any customer can report either fax issues or e-mail issues, so you need to have two different types of error handlers. Let us call them `EmailErrorHandler` and `FaxErrorHandler`. You can safely assume that `EmailErrorHandler` will handle e-mail errors only and will not fix fax errors. In the same manner, `FaxErrorHandler` will handle fax errors and will not care about e-mail errors.

You can form a chain like this: whenever your application finds an error, it will raise a ticket and forward the error with the hope that one of those handlers will handle it. Let's assume that the request first comes to `FaxErrorHandler`. If this handler agrees that it is a fax issue, it will handle it; otherwise, it will forward the issue to `EmailErrorHandler`.

Note that here the chain ends with `EmailErrorHandler`. But if you need to handle another type of issue, say, an authentication issue because of security vulnerabilities, you can make an `AuthenticationErrorHandler` and put it after `EmailErrorHandler` in the chain. Now if `EmailErrorHandler` also cannot fix the issue completely, it will forward the issue to `AuthenticationErrorHandler`, and the chain will end there.

Points To Remember

This is just an example; you are free to place these handlers in any order you'd like. The bottom line is that the processing chain may end in either of these two scenarios:

- A handler can process the request completely.
- You have reached the end of the chain.

Here is another example for you:

- Java's exception handling mechanism can be considered as a variation of this pattern. Consider a program that has a `try` block followed by multiple `catch` blocks. You know that each `catch` block is an exception handler that handles the type of exception indicated by its argument. But you cannot place them arbitrarily. To avoid the compile-time errors, you need to arrange these `catch` blocks from "more specific" to "more general" type. If there is no `catch` block to handle the exception, then the current thread (the thread that encountered the exception) is terminated. In this context, it is important to note that the on-line link <https://docs.oracle.com/javase/tutorial/essential/exceptions/catch.html> also says: "Exception handlers can do more than just print error messages or halt the program. They can do error recovery, prompt the user to make a decision, or propagate the error up to a higher-level handler using chained exceptions."

Implementation

In the following example, you'll write the program for the computer world example just discussed. In this example, you need to process different messages that may come from either e-mail or fax. The beginning of the program has the following segments of code to describe the error messages:

```
class Message {
    public String text;
    public Message(String msg) {
        text = msg;
    }
}
```

Then you use a handler interface that looks like the following:

```
interface Handler {
    void handleMessage(Message message);
    void nextErrorHandler(Handler nextReceiver);
}
```

The `FaxErrorHandler` and `EmailErrorHandler` classes implement this interface. They are the concrete handlers in this program. It is important to note that to demonstrate a very simple use case, you could use the following code segment in `FaxErrorHandler`:

```
if (message.text.contains("fax"))
{
    System.out.println("FaxErrorHandler processed the issue :" + message.text);
}
else if (nextReceiver != null)
{

```

```
nextReceiver.HandleMessage(message);  
}
```

Points To Remember

In the previous code segment, you can see that if a message contains the word “fax,” then `FaxErrorHandler` will handle it; otherwise, it will pass the issue to the next handler. Similarly, in the upcoming example, if a message contains the word “email,” then `EmailErrorHandler` will handle the message and so forth. But what happens if both “email” and “fax” are included in a message? I have taken care of this case in the upcoming example, but for simplicity, you can ignore the case using this segment of code.

In a real-world problem, one error may cause another error, so it is possible when an error occurs in the fax code base, the same error propagates to the email codebase (if they share a common codebase). As a result, a common fix can solve both issues. In the upcoming example, I show when you should be concerned about the issue and how to pass it to the next handler. So, in the beginning, you may ignore the individual pillar complexities.

Actually, an organization may prefer to implement an AI-based mechanism to analyze an issue first, and then based on the symptoms they can forward the issue to a particular department. But at the core, you may see the use of this pattern.

To demonstrate a case where a message contains both “email” and “fax,” let’s use a relatively complex structure for `FaxErrorHandler`. How does it work? Read the following points:

- If the error message contains the word “fax” but not “email,” after fixing the issue, it will not pass the error to the email handler.
- If the error message contains both the words “email” and “fax,” it will fix the fax issue and then it will pass it to the email handler so that the email team can verify the fix and can add additional code if they think it’s necessary.
- If the error message does not contain either “email” or “fax,” the issue will ultimately be forwarded to the unknown handler.

`EmailErrorHandler` and `FaxErrorHandler` are similar. Let’s pick one of them for your easy reference (the associated comments can be your guide):

```
class FaxErrorHandler implements Handler {  
    private Handler nextHandler;  
    @Override  
    public void nextErrorHandler(Handler nextHandler) {  
        this.nextHandler = nextHandler;  
    }  
}
```

```

@Override
public void handleMessage(Message message) {
    if (message.text.contains("fax")) {
        // Error message contains both- 'fax' and 'email'
        if (message.text.contains("email")) {
            System.out.println("-FaxErrorHandler
                fixed the fax issue: " + message.text);
            System.out.println("--Now
                EmailErrorHandler needs to cross verify it.");
            if (nextHandler != null) {
                nextHandler.handleMessage(message);
            }
        }
        // It's a fax error only.
    } else {
        System.out.println("FaxErrorHandler
            processed the issue: " + message.text);
    }

}

// Neither a fax issue nor an email issue
else {
    if (nextHandler != null) {
        nextHandler.handleMessage(message);
    }
}
}
}

```

Now if you have a message that contains both “email” and “fax” like “Neither fax nor email is working properly,” this relatively complex structure can help you to get the following output where you can see that both teams worked on the defect:

```

-FaxErrorHandler fixed the fax issue: Neither email nor fax is working properly.
--Now EmailErrorHandler needs to cross verify it.
-EmailErrorHandler fixed the email issue: Neither email nor fax is working properly.
--Now FaxErrorHandler needs to cross verify it.

```

At the end of this chain, there is an `UnknownErrorHandler` that simply says the issue is neither from `Email` nor from `Fax`, so consult the expert developers to tackle the issue.

```

class UnknownErrorHandler implements Handler {
    private Handler nextHandler;
    @Override
    public void setNextHandler(Handler nextHandler) {
        this.nextHandler = nextHandler;
    }
    @Override
    public void handleMessage(Message message) {
        if (!(message.text.contains("fax") || message.text.contains("email"))

```

```

        System.out.println("Unknown error occurs.Consult experts immed:");
    } else if (nextHandler != null) {
        nextHandler.handleMessage(message);
    }

    }
}

```

The forming of the error handler objects is quite straightforward. From the following code segment and supportive comments, you can easily understand how to set a starting point and make a chain of handlers:

```

// Set the root handler/ starting point
Handler rootHandler=faxHandler;
// Making the following chain:
// FaxErrorHandler->EmailErrorHandler->UnknownHandler->End.
rootHandler.nextErrorHandler(emailHandler);
emailHandler.nextErrorHandler(unknownHandler);
unknownHandler.nextErrorHandler(null);

```

Why use `rootHandler`? If you form a new chain by putting these handlers in a different order, you do not need to make any change inside the client code, which is as follows (you just need to tell which one is `rootHandler` at that point):

```

System.out.println("Handling different types of errors.");
System.out.println("\nError-1:");
rootHandler.handleMessage(msg1);
System.out.println("\nError-2:");
rootHandler.handleMessage(msg2);
System.out.println("\nError-3:");
rootHandler.handleMessage(msg3);
System.out.println("\nError-4:");
rootHandler.handleMessage(msg4);
System.out.println("\nError-5:");
rootHandler.handleMessage(msg5);
System.out.println("\nError-6:");
rootHandler.handleMessage(msg6);

```

Class Diagram

Figure [17-2](#) shows the class diagram.

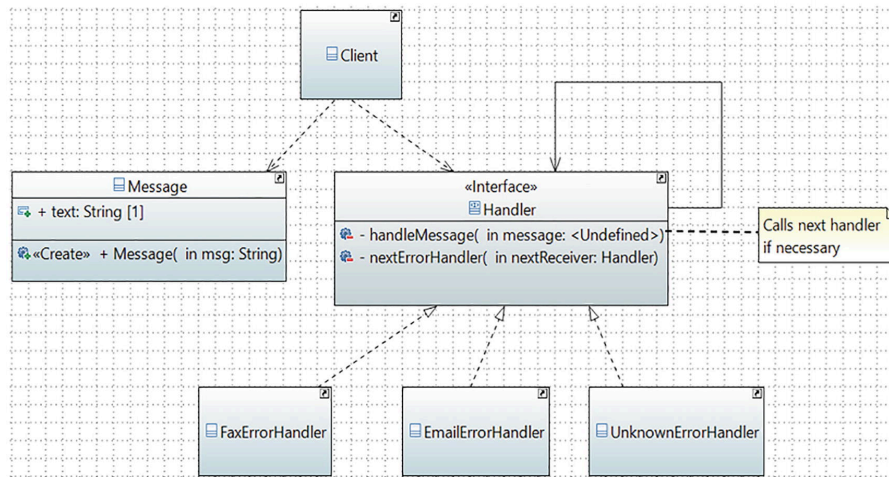


Figure 17-2 Class diagram

Package Explorer View

Figure 17-3 shows the high-level structure of the program.

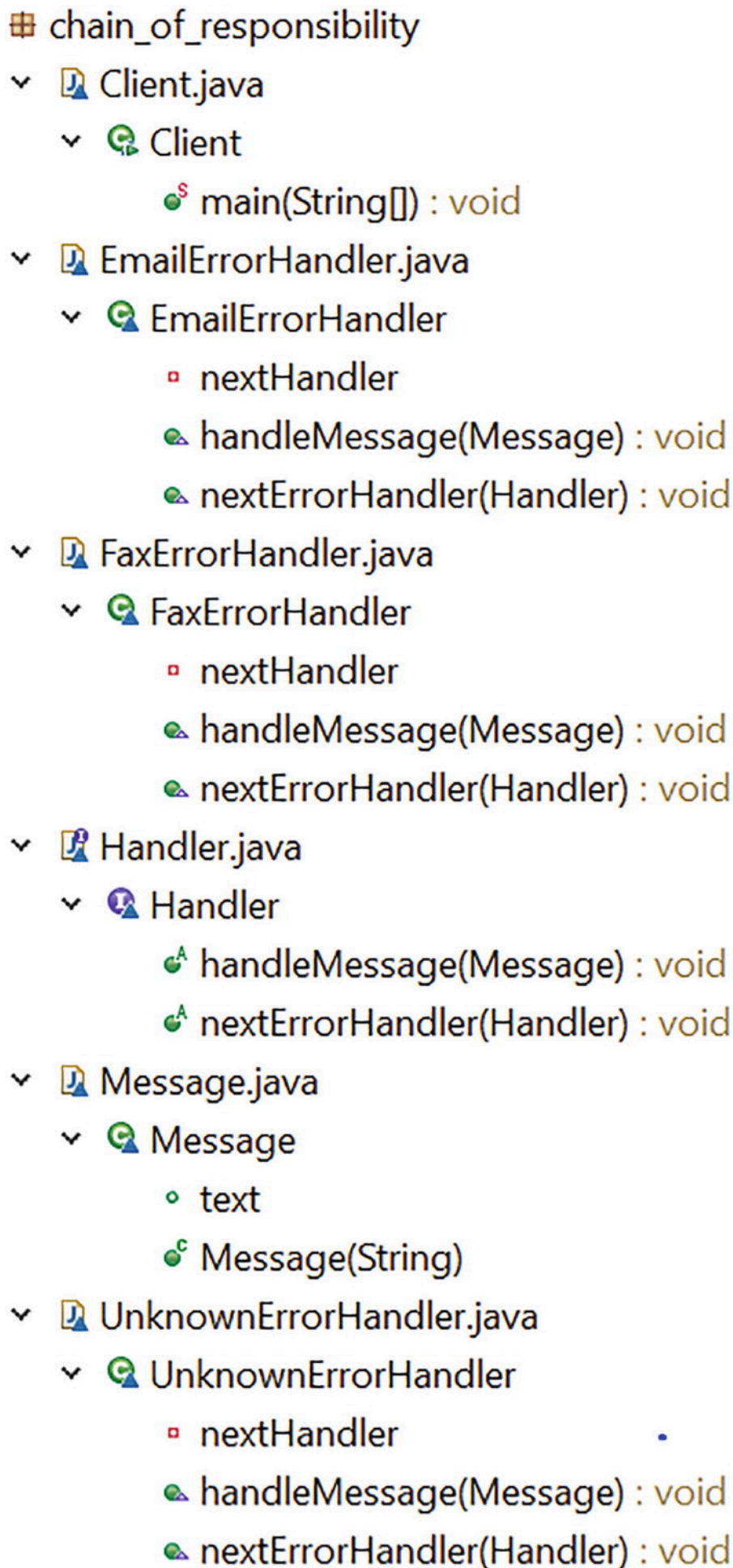


Figure 17-3 Package Explorer view

Demonstration

Here's the complete program. All parts of the program are stored inside the package `jdp3e.chain_of_responsibility`.

```
// Message.java
class Message {
    public String text;
    public Message(String msg) {
        text = msg;
    }
}

// Handler.java
interface Handler {
    void handleMessage(Message message);
    void nextErrorHandler(Handler nextHandler);
}

// FaxErrorHandler.java

class FaxErrorHandler implements Handler {
    private Handler nextHandler;
    @Override
    public void nextErrorHandler(Handler nextHandler) {
        this.nextHandler = nextHandler;
    }
    @Override
    public void handleMessage(Message message) {
        if (message.text.contains("fax")) {
            // Error message contains both- 'fax' and 'email'
            if (message.text.contains("email")) {
                System.out.println("-FaxErrorHandler
                    fixed the fax issue: " + message.text);
                System.out.println("--Now
                    EmailErrorHandler needs to cross verify it.");
                if (nextHandler != null) {
                    nextHandler.handleMessage(message);
                }
            }
            // It's a fax error only.
            else {

                System.out.println ("FaxErrorHandler
                    processed the issue: " + message.text);
            }
        }
        // Neither a fax issue nor an email issue
        else {
            if (nextHandler != null) {
                nextHandler.handleMessage(message);
            }
        }
    }
}
```

```

    }
}
// ErrorHandler.java
class ErrorHandler implements Handler {
    private Handler nextHandler;
    @Override
    public void nextErrorHandler(Handler nextHandler) {
        this.nextHandler = nextHandler;
    }
    @Override
    public void handleMessage(Message message) {
        if (message.text.contains("email")) {
            // Error message contains both-'email' and 'fax'
            if (message.text.contains("fax")) {
                System.out.println("-ErrorHandler
                    fixed the email issue: " +
                    message.text);
                System.out.println("--Now
                    FaxErrorHandler needs to cross verify it.");
                if (nextHandler != null) {
                    nextHandler.handleMessage(message);
                }
            }

            // It's an email error only.
            else {
                System.out.println("ErrorHandler
                    processed the issue: "+ message.text);
            }
        }
        // Neither a fax issue nor an email issue
        else {
            if (nextHandler != null) {
                nextHandler.handleMessage(message);
            }
        }
    }
}
// UnknownErrorHandler.java

```

```

class UnknownErrorHandler implements Handler {
    private Handler nextHandler;
    @Override
    public void nextErrorHandler(Handler nextHandler) {
        this.nextHandler = nextHandler;
    }
    @Override
    public void handleMessage(Message message) {
        if (!(message.text.contains("fax") ||
            message.text.contains("email")))

```

```

        ) {
            System.out.println("An unknown error
                                occurs. Consult experts immediately.");
        } else if (nextHandler != null) {
            nextHandler.handleMessage(message);
        }
    }
}

// Client.java
class Client {

    public static void main(String[] args) {
        System.out.println("***Chain of Responsibility Pattern Demo***\n");
        // Objects of the chains
        Handler faxHandler = new FaxErrorHandler();
        Handler emailHandler = new EmailErrorHandler();
        Handler unknownHandler = new UnknownErrorHandler();
        // The root handler/ starting point
        Handler rootHandler;
        // Making the following chain:
        // FaxErrorHandler->EmailErrorHandler-
        // >UnknownHandler->End.
        rootHandler = faxHandler;
        rootHandler.nextErrorHandler(emailHandler);
        emailHandler.nextErrorHandler(unknownHandler);
        unknownHandler.nextErrorHandler(null);
        // Different error messages

        Message msg1 = new Message("The fax is going
slow.");
        Message msg2 = new Message("The emails are not reaching
                                destinations.");
        Message msg3 = new Message("Sometimes the BCC field
                                is disabled in
emails.");
        Message msg4 = new Message("The fax is not reaching destinations.");
        Message msg5 = new Message("Neither email nor fax is working
                                properly.");
        Message msg6 = new Message("Users cannot login into the system.");
        System.out.println("Handling different types of errors.");
        System.out.println("\nError-1:");
        rootHandler.handleMessage(msg1);
        System.out.println("\nError-2:");
        rootHandler.handleMessage(msg2);
        System.out.println("\nError-3:");
        rootHandler.handleMessage(msg3);
        System.out.println("\nError-4:");
        rootHandler.handleMessage(msg4);
        System.out.println("\nError-5:");
        rootHandler.handleMessage(msg5);
    }
}

```

```

        System.out.println("\nError-6:");
        rootHandler.handleMessage(msg6);
    }
}

```

Output

Here's the output:

```

***Chain of Responsibility Pattern Demo***
Handling different types of errors.
Error-1:
FaxErrorHandler processed the issue: The fax is going slow.
Error-2:
EmailErrorHandler processed the issue: The emails are not reaching destination
Error-3:
EmailErrorHandler processed the issue: Sometimes the BCC field is disabled in
Error-4:
FaxErrorHandler processed the issue: The fax is not reaching destinations.
Error-5:
-FaxErrorHandler fixed the fax issue: Neither email nor fax is working properly.
--Now EmailErrorHandler needs to cross verify it

.
-EmailErrorHandler fixed the email issue: Neither email nor fax is working properly.
--Now FaxErrorHandler needs to cross verify it.
Error-6:
An unknown error occurs. Consult experts immediately.

```

Q&A Session

17.1 What are the advantages of using the Chain of Responsibility design pattern?

Some notable advantages are as follows:

- You have more than one object to handle a request. (Notice that if a handler cannot handle the whole request, it can forward the responsibility to the next handler in the chain.)
- The nodes of the chain can be added or removed dynamically. Also, you can shuffle their order. For example, in the previous application, if you see that most issues are e-mail issues, then you may place `EmailErrorHandler` as the first handler to save the average processing time of the application.
- A handler does not need to know how the next handler in the chain will handle the request. It can focus on its own handling mechanism only.

- In this pattern, you are decoupling the senders (of requests) from the receivers.

17.2 What are the challenges associated with using the Chain of Responsibility design pattern?

The following points talk about some challenges:

- There is no guarantee that the request will be handled because you may reach the end of the chain but not found any explicit receiver to handle the request. (I agree that depending upon the situation, this can be an advantage too. Consider the case when the Java exception handlers handle an exception on behalf of you.)
- Debugging becomes tricky with this kind of design.
- If there are many handlers, you need to be careful about placing them in the proper order to improve the overall response time.

17.3 How can you handle the scenario where you have reached the end of the chain but no handler handled the request?

One simple solution is through `try-catch` (or `try-finally` or `try-catch-finally`) blocks. If there is no handler to handle the request, you can raise an exception with the appropriate messages and catch the exception in the `catch` block to draw attention to it (or handle it in a different way).

The GoF talked about Smalltalk's automatic forwarding mechanism (`doesNotUnderstand`) in a similar context. If a message cannot find a proper handler, it will be caught in `doesNotUnderstand` implementations that can be overridden to forward the message in the object's successor, log it in some file, and store it in a queue for later processing, or you can simply perform any other intended operations. But note that, by default, this method raises an exception that needs to be handled properly.

17.4 In short, I can say that a handler either handles the request fully or passes it to the next handler. Is this correct?

Yes.

17.5 Can you give me a demo of making a different chain?

See the following code block with supporting comments:

```
// Making a different chain:
// EmailErrorHandler->FaxErrorHandler->UnknownHandler->End.
rootHandler = emailHandler;
rootHandler.nextErrorHandler(faxHandler);
faxHandler.nextErrorHandler(unknownHandler);
unknownHandler.nextErrorHandler(null);
```

```

once you run the code now, you'll see the following output:
***Chain of Responsibility Pattern Demo***
Handling different types of errors.
Error-1:
FaxErrorHandler processed the issue: The fax is going slow.
Error-2:
EmailErrorHandler processed the issue: The emails are not reaching destination
Error-3:
EmailErrorHandler processed the issue: Sometimes the BCC field is disabled in
Error-4:
FaxErrorHandler processed the issue: The fax is not reaching destinations.
Error-5:
-EmailErrorHandler fixed the email issue: Neither email nor fax is working pro
--Now FaxErrorHandler needs to cross verify it.
-FaxErrorHandler fixed the fax issue: Neither email nor fax is working proper
--Now EmailErrorHandler needs to cross verify it

.
Error-6:
An unknown error occurs. Consult experts immediately.

```

Notice inside Error-5 now the email issue is fixed first and then the fax issue is fixed. This is the power of this pattern. You can shuffle your handlers as per your need.

17.6 In your demonstration, a Message instance contains a String. So, it appears to me, instead of using the Message instances, you could use simple strings inside the client code to reduce the overall code size. Am I right?

Nice observation. You are correct. But I want you to note that a message instance may contain different things, such as ids and priorities. In the previous version of the book, I used message priorities. This time I made the implementation simpler, but I want you to follow the structure when you need to include additional information in an error message. This comment applies to all similar implementations in this book.

17.7 It appears to me that there are similarities between the Observer pattern (Chapter 16) and the Chain of Responsibility pattern. Is this correct?

In an Observer pattern, all registered users can get notifications at the same time, but in the case of the Chain of Responsibility pattern, objects in the chain are notified one by one, which can happen until an object handles the notification fully. Figure 17-4 and Figure 17-5 summarize the difference.

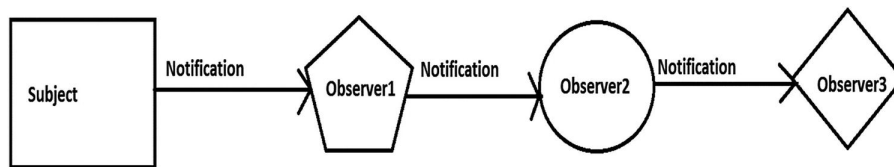


Figure 17-4 Chain of Responsibility pattern

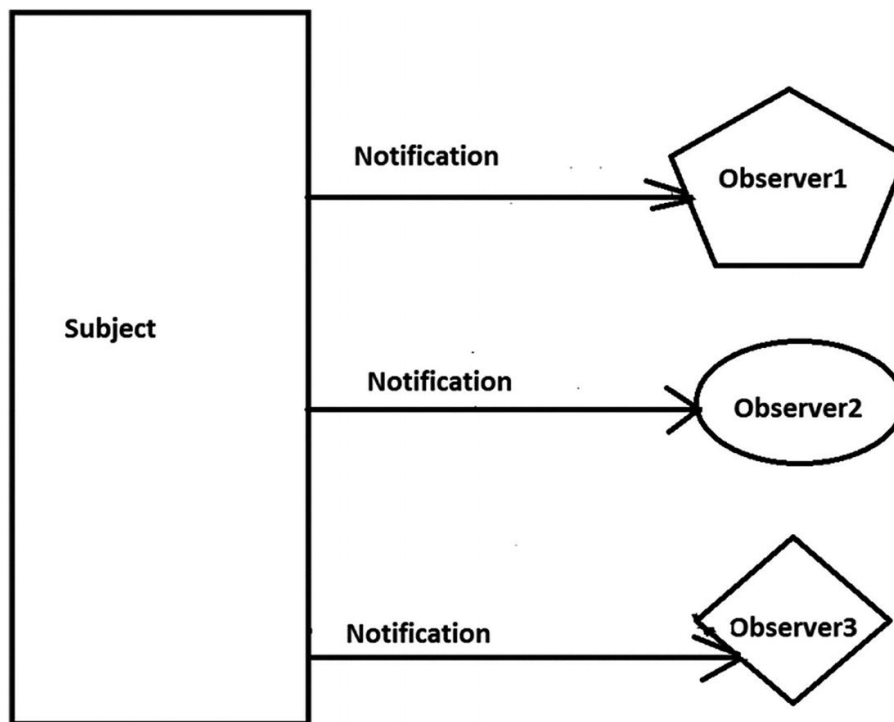


Figure 17-5 Observer pattern

In Figure [17-4](#), I assume that Observer3 was able to process the notification completely. So, it is the end node of the chain.

Previous chapter

< [16. Observer Pattern](#)

Next chapter

[18. Iterator Pattern](#) >